NAME: K. SWARNA VARSHINI

REG.NO: 192324081

SIMATS ENGINEERING

ASSESSMENT TOOL 4

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1715

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3 **Duration: 1 hour**

Total Marks: 30

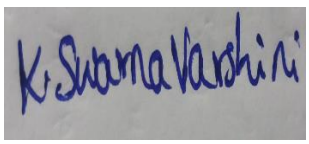
Annexure A

Application Based Questions

Code of Conduct:

I _K. Swarna Varshini__(192324081)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

Application Based Questions

1. Intelligent Route Planning using A* and Greedy Search.

A smart delivery system is designed to transport medical supplies between cities. The system

uses informed search strategies to determine optimal routes.

The road network is given below:

City	Connected To	Cost	Heuristic h(n) to Goal (G)
A	B, C	2, 4	7
B	D, E	3, 6	6
C	F	5	4
D	G	4	3
E	G	2	2
F	G	3	1
G	—	—	0

Tasks:

i. Construct the search tree from A to G.

ii. Apply Greedy Best-First Search and show node expansion order.

iii. Apply A* algorithm using .

iv. Justify which algorithm is better for real-time emergency delivery systems.

2. Constraint Satisfaction Problem – University Timetable Scheduling

A university must schedule final exams for 5 subjects: AI, ML, DBMS, OS, CN across 3 days

(Day 1, Day 2, Day 3) with the conditions, AI and ML cannot be scheduled on the same day,

DBMS must be scheduled before OS, CN cannot be on Day 1, At most 2 exams per day, A

student is enrolled in AI, DBMS, and CN → these cannot overlap

Tasks:

i. Formulate this problem as a Constraint Satisfaction Problem (CSP) by defining,

Variables, Domains and Constraints

ii. Draw the constraint graph.

iii. Solve using Backtracking Search.

iv. Apply Forward Checking or Arc Consistency and explain how it reduces the search space.

v. Provide one valid timetable solution.

3. Game Playing – Minimax and Alpha-Beta Pruning

An AI agent is designed to play a turn-based strategy game. The game tree is given below:

Tasks:

i. Apply the Minimax algorithm and compute values for all internal nodes.

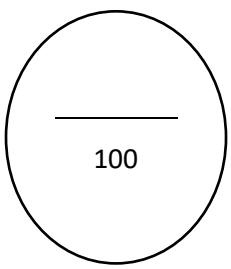
ii. Identify the optimal move for MAX.

iii. Apply Alpha-Beta Pruning to Show α and β values at each step and Clearly indicate which nodes are pruned

iv. Compare nodes evaluated in Minimax vs Alpha-Beta

v. Explain the importance of evaluation functions in complex games like chess.

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total Marks (30)
Problem Understanding	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Constraint Analysis	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Solution Development	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Application of Concepts	15% (1.5)	/1.5	/1.5	/1.5	/4.5
Justification	10% (1)	/1	/1	/1	/3
Total per Question	**10 Marks**	**/10**	**/10**	**/10**	**/30**

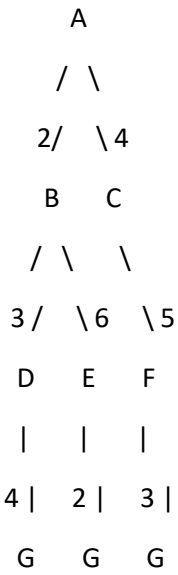


ANSWERS

1. Intelligent Route Planning using A* and Greedy Search

i. Construct the Search Tree from A to G

The search tree is:



Possible paths are:

1. **A → B → D → G**
    - Cost = 2 + 3 + 4 = **9**
2. **A → B → E → G**
    - Cost = 2 + 6 + 2 = **10**
3. **A → C → F → G**
    - Cost = 4 + 5 + 3 = **12**

Therefore, the **optimal path is A → B → D → G with cost 9**.

ii. Greedy Best-First Search

Greedy Best-First Search selects the node with the **smallest heuristic value h(n)**.

Step 1: Start at A

A

Children:

- B → h(B) = 6
- C → h(C) = 4

Choose **C**, because 4 < 6.

Step 2: Expand C

C has:

- F → h(F) = 1

Choose **F**.

Step 3: Expand F

F has:

- G → h(G) = 0

Choose **G**.

Node Expansion Order

A → C → F → G

Path found

A → C → F → G

Total Cost

$$4 + 5 + 3 = \boxed{12}$$

Greedy Search Result

Route: A → C → F → G
Cost: 12

Greedy search reaches the goal quickly, but it does **not guarantee the shortest path**.

---

iii. Apply A* Algorithm

A* uses:

$$\boxed{f(n) = g(n) + h(n)}$$

Where:

- $g(n)$ = actual cost from A to node n
- $h(n)$ = estimated cost from n to G
- $f(n)$ = total estimated cost

Step 1: Expand A

For B:

$$g(B) = 2$$
$$f(B) = 2 + 6 = 8$$

For C:

$$g(C) = 4$$
$$f(C) = 4 + 4 = 8$$

Node	g(n)	h(n)	f(n)=g+h
B	2	6	**8**
C	4	4	**8**

There is a tie. We can select **B first**.

Step 2: Expand B

B connects to D and E.

For D:

$$g(D) = 2 + 3 = 5$$
$$f(D) = 5 + 3 = 8$$

For E:

$$g(E) = 2 + 6 = 8$$
$$f(E) = 8 + 2 = 10$$

Node	g	h	f
C	4	4	**8**
D	5	3	**8**
E	8	2	10

Again, there is a tie between C and D. Select **C** first.

Step 3: Expand C

C connects to F.

$$g(F) = 4 + 5 = 9$$
$$f(F) = 9 + 1 = 10$$

Node	g	h	f
D	5	3	**8**
E	8	2	10
F	9	1	10

Select **D**.

Step 4: Expand D

D connects to G.

$$g(G) = 5 + 4 = 9$$
$$f(G) = 9 + 0 = \boxed{9}$$

Since G is the goal, A* terminates.

A* Expansion Order

A → B → C → D → G

Optimal Path

A → B → D → G

Total Cost

$$2 + 3 + 4 = \boxed{9}$$

iv. Comparison and Justification

Feature	Greedy Best-First Search	A* Search
Evaluation	h(n)	g(n) + h(n)
Expansion order	A → C → F → G	A → B → C → D → G
Path	A → C → F → G	A → B → D → G
Total cost	**12**	**9**
Optimal path?	No	**Yes**
Uses actual path cost?	No	**Yes**
Suitable for emergency delivery?	Less suitable	**More suitable**

Conclusion

A* is better for a real-time emergency medical delivery system because it considers both:

- the **actual distance/cost already travelled**, $g(n)$
- the **estimated remaining cost**, $h(n)$

Thus, A* finds the **optimal route A → B → D → G with cost 9**, whereas Greedy Best-First Search chooses a faster-looking route based only on the heuristic and produces a more expensive route of cost 12.

Therefore:

A* Search is preferred for emergency delivery route planning.

2. Constraint Satisfaction Problem – University Timetable Scheduling

We have **5 subjects**:

- AI
- ML
- DBMS
- OS
- CN

and **3 days**:

- Day 1
- Day 2
- Day 3

---

i. Formulate the Problem as a CSP

A **Constraint Satisfaction Problem (CSP)** consists of:

1. Variables

Each subject is a variable representing the day on which its exam is scheduled.

$$X = \{A, I, M, L, DBMSOSCN\}$$

So,

- AI = day of AI exam
- ML = day of ML exam
- $DBMS$ = day of DBMS exam
- OS = day of OS exam
- CN = day of CN exam

---

2. Domains

Each exam can initially be scheduled on any of the 3 days:

$$D(AI) = D(ML) = D(DBMS) = D(OS) = D(CN)$$
$$= \{D, 1, D2D3\}$$

However, because CN cannot be on Day 1:

$$D(CN) = \{D, 2D3\}$$

---

3. Constraints

Constraint 1: AI and ML cannot be on the same day

$$AI \neq ML$$

Constraint 2: DBMS must be scheduled before OS

$$Day(DBMS) < Day(OS)$$

Therefore, possible combinations are:

$$(D, BMSOS) = (D, 1D2), (D, 1D3), (D, 2D3)$$

Constraint 3: CN cannot be on Day 1

$$CN \neq D1$$

Constraint 4: At most 2 exams per day

For every day:

$$Count(D1) \leq 2$$
$$Count(D2) \leq 2$$
$$Count(D3) \leq 2$$

Constraint 5: AI, DBMS and CN cannot overlap

Since one student is enrolled in all three subjects:

$$AI \neq DBMS$$
$$AI \neq CN$$
$$DBMS \neq CN$$

CSP Representation

$Variables$: $\{A, I, M, L, DBMSOSCN\}$
$Domains$: $\{D, 1, D2D3\}$
$Constraints$: $AI \neq ML$
$DBMS < OS$
$CN \neq D1$
$AI \neq DBMS$
$AI \neq CN$
$DBMS \neq CN$
Maximum 2 exams/day

---

ii. Constraint Graph

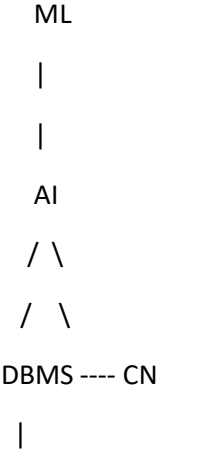
In a constraint graph:

- **Nodes** = subjects
- **Edges** = constraints between subjects

The main pairwise constraints are:

- AI ↔ ML
- AI ↔ DBMS
- AI ↔ CN
- DBMS ↔ CN
- DBMS → OS

The graph can be represented as:



|

OS

Edge meanings

AI —— ML → AI ≠ ML

AI —— DBMS → AI ≠ DBMS

AI —— CN → AI ≠ CN

DBMS —— CN → DBMS ≠ CN

DBMS —→ OS → DBMS must be before OS

Additionally:

CN ≠ Day 1

Maximum 2 exams per day

These are **unary/global constraints**, so they are not represented as ordinary subject-to-subject edges.

---

iii. Solve Using Backtracking Search

Backtracking assigns a day to each subject and checks the constraints after every assignment.

We can choose the variables in this order:

$$DBMS \rightarrow OS \rightarrow AI \rightarrow CN \rightarrow ML$$

Step 1: Assign DBMS

Choose:

$$DBMS = D1$$

Since DBMS must be before OS:

$$OS \in \{D, 2D3\}$$

Step 2: Assign OS

Choose:

$$OS = D2$$

Now:

DBMS = D1

OS = D2

Step 3: Assign AI

AI cannot be on the same day as DBMS because the student has both exams.

Therefore:

$$AI \neq D1$$

Choose:

$$AI = D2$$

This is allowed because AI and OS do not have a conflict.

Step 4: Assign CN

CN cannot be Day 1.

Also:

$$CN \neq AI$$
$$CN \neq DBMS$$

Since:

- AI = D2
- DBMS = D1

CN cannot be D1 or D2.

Therefore:

$$CN = D3$$

Step 5: Assign ML

AI and ML cannot be on the same day.

Since:

$$AI = D2$$

ML cannot be D2.

Choose:

$$ML = D1$$

Final assignment

DBMS = D1

ML = D1

AI = D2

OS = D2

CN = D3

Check the maximum:

- Day 1 → 2 exams
- Day 2 → 2 exams
- Day 3 → 1 exam

All constraints are satisfied.

---

iv. Forward Checking / Arc Consistency

Forward Checking

Forward checking removes values from the domains of unassigned variables whenever a value is assigned.

Initially:

Subject	Domain
AI	D1, D2, D3
ML	D1, D2, D3
DBMS	D1, D2, D3
OS	D1, D2, D3
CN	D2, D3

Assign DBMS = D1

Because:

$$DBMS < OS$$

OS cannot be D1.

So:

$$D(OS) = \{D, 2D3\}$$

Because DBMS and CN cannot overlap:

$$D(CN) = \{D, 2D3\}$$

Because DBMS and AI cannot overlap:

$$D(AI) = \{D, 2D3\}$$

Now:

Subject	Reduced Domain
AI	D2, D3
ML	D1, D2, D3
DBMS	**D1**
OS	D2, D3
CN	D2, D3

Already several possibilities have been eliminated.

Assign OS = D2

Now AI and CN cannot overlap with their conflicting subjects where applicable.

If we assign:

$$AI = D2$$

then:

$$D(ML) = \{D, 1D3\}$$

because AI and ML cannot be on the same day.

Also:

$$D(CN) = \{D3\}$$

because CN cannot overlap with AI or DBMS.

Thus:

AI = D2

CN = D3

ML = D1 or D3

Choose:

$$ML = D1$$

How Forward Checking Reduces Search Space

Without forward checking, every subject initially has approximately 3 possible values:

$$3^5 = 243$$

possible assignments before considering constraints.

Forward checking immediately removes invalid values after every assignment.

For example:

DBMS = D1

↓

OS cannot D1

AI cannot D1

CN cannot D1

↓

Domains become smaller

↓

Fewer assignments need to be explored

Therefore, **Forward Checking avoids exploring many invalid branches** and makes backtracking faster.

v. One Valid Timetable Solution

Day	**Exams**
Day 1	DBMS, ML
Day 2	AI, OS
Day 3	CN

Verification

1. AI and ML cannot be same day:

$$AI = D2,\ ML = D1$$

✓ Satisfied

2. DBMS before OS:

$$DBMS = D1,\ OS = D2$$

$$D1 < D2$$

✓ Satisfied

3. CN cannot be Day 1:

$$CN = D3$$

✓ Satisfied

4. Maximum 2 exams per day:

- Day 1 → 2
- Day 2 → 2
- Day 3 → 1

✓ Satisfied

5. AI, DBMS and CN cannot overlap:

$$AI = D2,\ DBMS = D1,\ CN = D3$$

All are on different days.

✓ Satisfied

Final Answer

Day	Exams
$D1$	$DBMS, ML$
$D2$	AI, OS
$D3$	CN

Backtracking solution:

$$\boxed{DBMS = D1, OS = D2, AI = D2, CN = D3, ML = D1}$$

Forward Checking: Reduces the domains after each assignment, eliminating invalid choices early and thereby reducing the search space.

3. Game Playing – Minimax and Alpha-Beta Pruning

Given the game tree, the leaf values from left to right are:

$$84,\ -29,\ -37,\ -25,\ 1,\ -43,\ -75,\ 49$$

The levels are:

- Root = **MIN**

- Next level = **MAX**
- Next level = **MIN**
- Bottom level = utility/leaf values

---

i. Apply Minimax Algorithm

Step 1: Evaluate the MIN nodes

For the first MIN node:

$$\min(84, -29) = -29$$

For the second MIN node:

$$\min(-, 37 - 25) = -37$$

For the third MIN node:

$$\min(1, -43) = -43$$

For the fourth MIN node:

$$\min(-, 7549) = -75$$

So:

Step 2: Evaluate the MAX nodes

Left MAX:

$$\max(-, 29 - 37) = \boxed{-29}$$

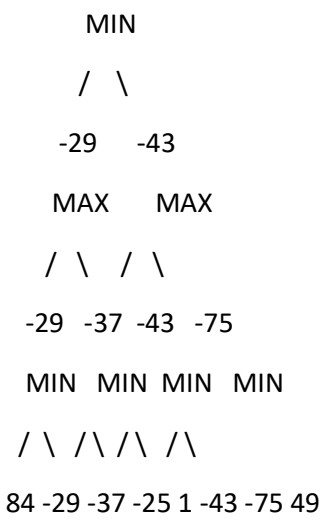
Right MAX:

$$\max(-, 43 - 75) = \boxed{-43}$$

Step 3: Evaluate the root MIN node

$$\min(-, 29 - 43) = \boxed{-43}$$

Complete Minimax Tree



Final Minimax Value

$$\boxed{-43}$$

---

ii. Identify the Optimal Move for MAX

At the **right MAX node**:

$$\max(-, 43 - 75) = -43$$

At the **left MAX node**:

$$\max(-, 29 - 37) = -29$$

Therefore, if MAX is choosing between these two branches:

$$\max(-, 29 - 43) = \boxed{-29}$$

So **MAX prefers the left branch**, because:

$$\boxed{-29 > -43}$$

However, notice that the **root of the given tree is MIN**, so the actual player at the root chooses the **right branch**, since MIN wants the smaller value:

$$\boxed{-43}$$

Thus:

- **MAX's preferred branch:** Left branch → **–29**
- **Actual root MIN move:** Right branch → **–43**

---

iii. Alpha-Beta Pruning

Alpha-Beta pruning uses two values:

- **α (Alpha):** best value MAX can guarantee so far
- **β (Beta):** best value MIN can guarantee so far

We evaluate the tree **from left to right**.

Initially:

$$\alpha = -\infty,\ \beta = +\infty$$

Step 1: Evaluate Left MAX Branch

At left MAX:

$$\alpha = -\infty,\ \beta = +\infty$$

First MIN node

Leaves:

$$84,\ -29$$

MIN chooses:

$$\min(84, -29) = -29$$

So at MAX:

$$\alpha = \max(-\infty, -29) = -29$$

Second MIN node

Now:

$$\alpha = -29,\ \beta = +\infty$$

First leaf:

$$-37$$

So:

$$\beta = -37$$

Now:

$$\beta \leq \alpha$$
$$-37 \leq -29$$

Therefore, the remaining leaf **−25 is pruned**.

Second MIN:

MIN

/ \

-37 -25

✓ ✗ PRUNED

Left MAX value:

$$\boxed{-29}$$

Step 2: Update Root MIN

Root receives:

$$-29$$

Therefore:

$$\beta = -29$$

Step 3: Evaluate Right MAX Branch

At right MAX:

$$\alpha = -\infty,\ \beta = -29$$

First MIN node

Leaves:

$$1,\ -43$$

MIN chooses:

$$\min(1, -43) = -43$$

MAX updates:

$$\alpha = -43$$

Second MIN node

Now:

$$\alpha = -43,\ \beta = -29$$

First leaf:

$$-75$$

So:

$$\beta = -75$$

Now:

$$\beta \leq \alpha$$
$$-75 \leq -43$$

Therefore, **49 is pruned**.

Second MIN:

MIN

/ \

-75 49

✓ ✗ PRUNED

Right MAX value:

$$\boxed{-43}$$

Final Root Calculation

Root is MIN:

$$\min(-, 29 - 43)$$
$$\boxed{-43}$$

Alpha-Beta Pruning Diagram

MIN

α=-∞ β=-43

/ \

MAX MAX

-29 -43

/ \ / \

MIN MIN MIN MIN

-29 -37 -43 -75

/ \ / \ / \ / \

84 -29 -37 -25 1 -43 -75 49

✗ ✗

PRUNED PRUNED

Pruned Nodes

$$\boxed{-25 \text{ and } 49}$$

iv. Comparison: Minimax vs Alpha-Beta

The complete tree contains:

- **8 leaf nodes**
- **7 internal nodes**
- **15 total nodes**

Minimax

Minimax evaluates all:

$$\boxed{15 \text{ nodes}}$$

Alpha-Beta

Alpha-Beta prunes:

$$-25,\ 49$$

So it evaluates:

$$15 - 2 = \boxed{13 \text{ nodes}}$$